

ENSF 380

TOPIC 16: UML CLASS RELATIONSHIPS

Relationships Between Classes

Association

Aggregation

Composition

Dependency

Association

Expresses a semantic connection between classes

- There is no hierarchy

Two classes are weakly connected

- No ownership
- No whole-part relationship

Unidirectional Association

One side is aware of the relationship



Bidirectional Association

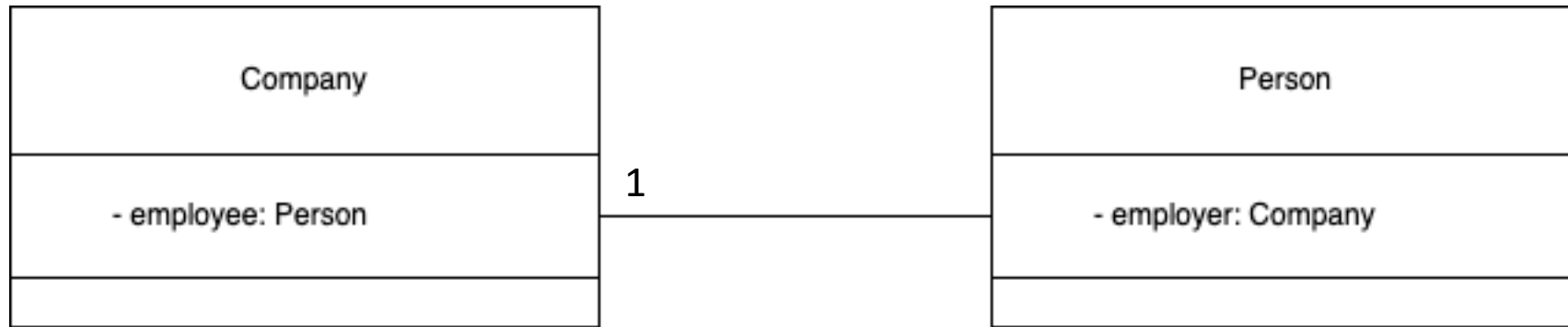
Both sides are aware of the relationship

- Use with caution, can easily create unmaintainable code



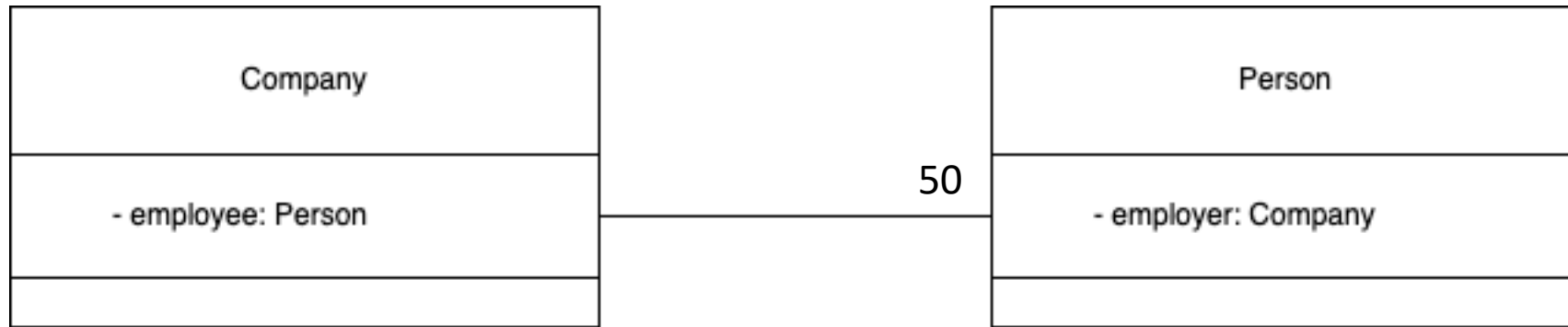
Cardinality

A Person is an employee at exactly one Company



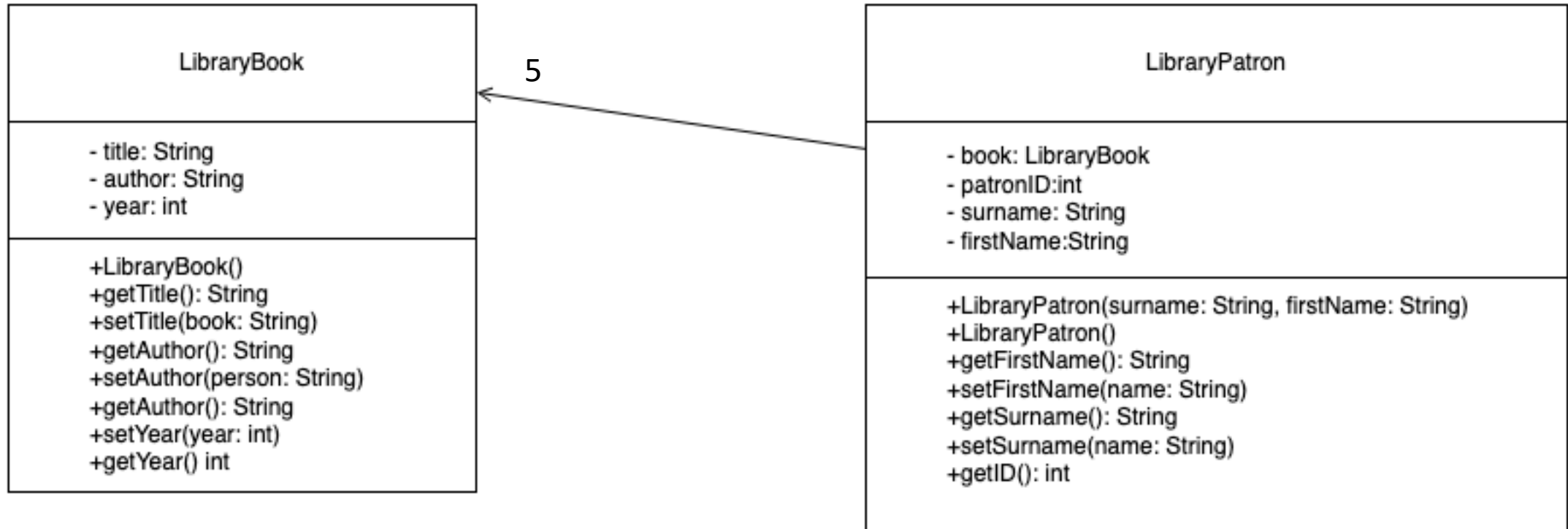
Cardinality

A Company employs exactly 50 People



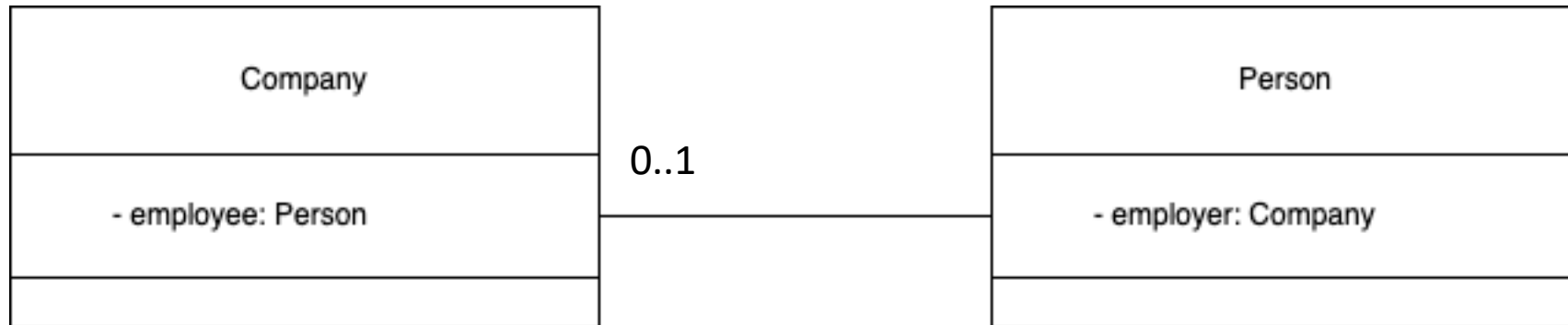
Cardinality

A LibraryPatron is using exactly 5 LibraryBooks



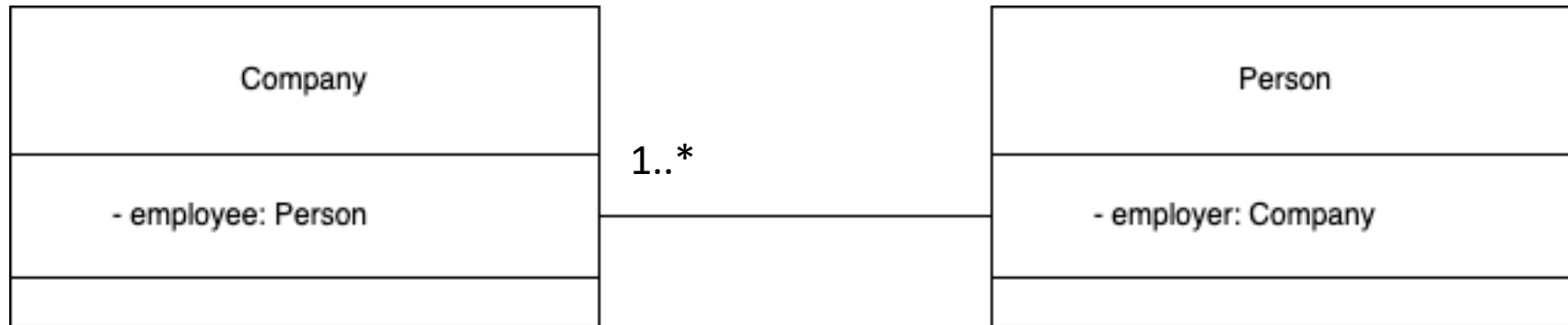
Cardinality

A Person may be employed by zero or one Companies



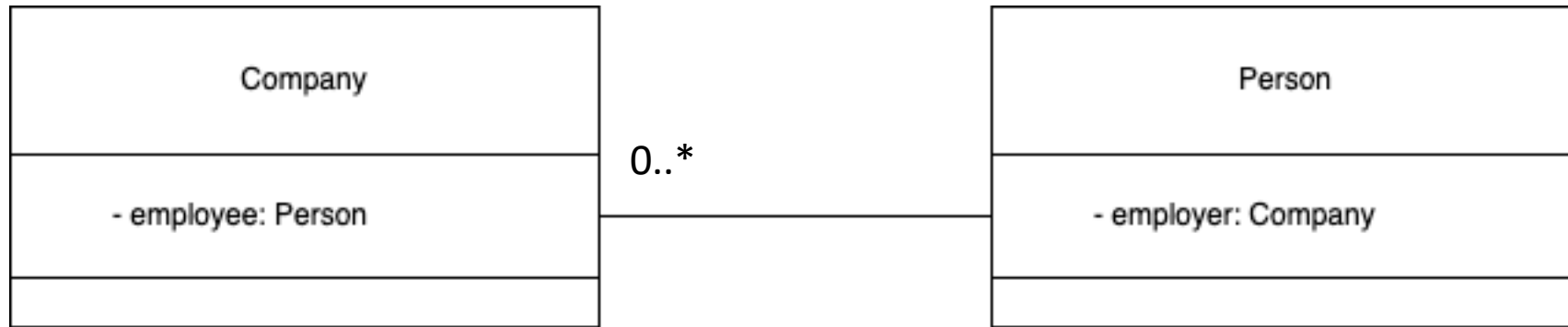
Cardinality

A Person must be employed by one or more Companies



Cardinality

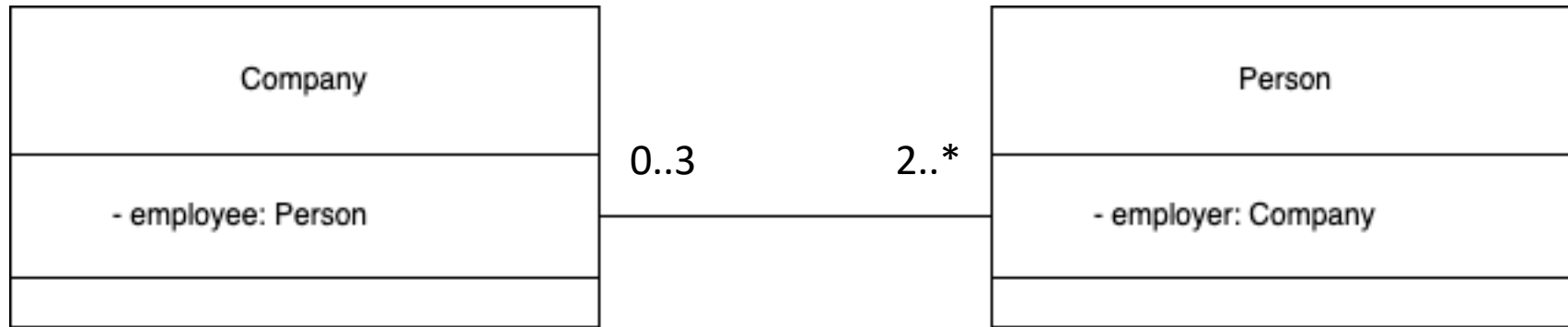
A Person may be employed by any number of Companies (zero or more)



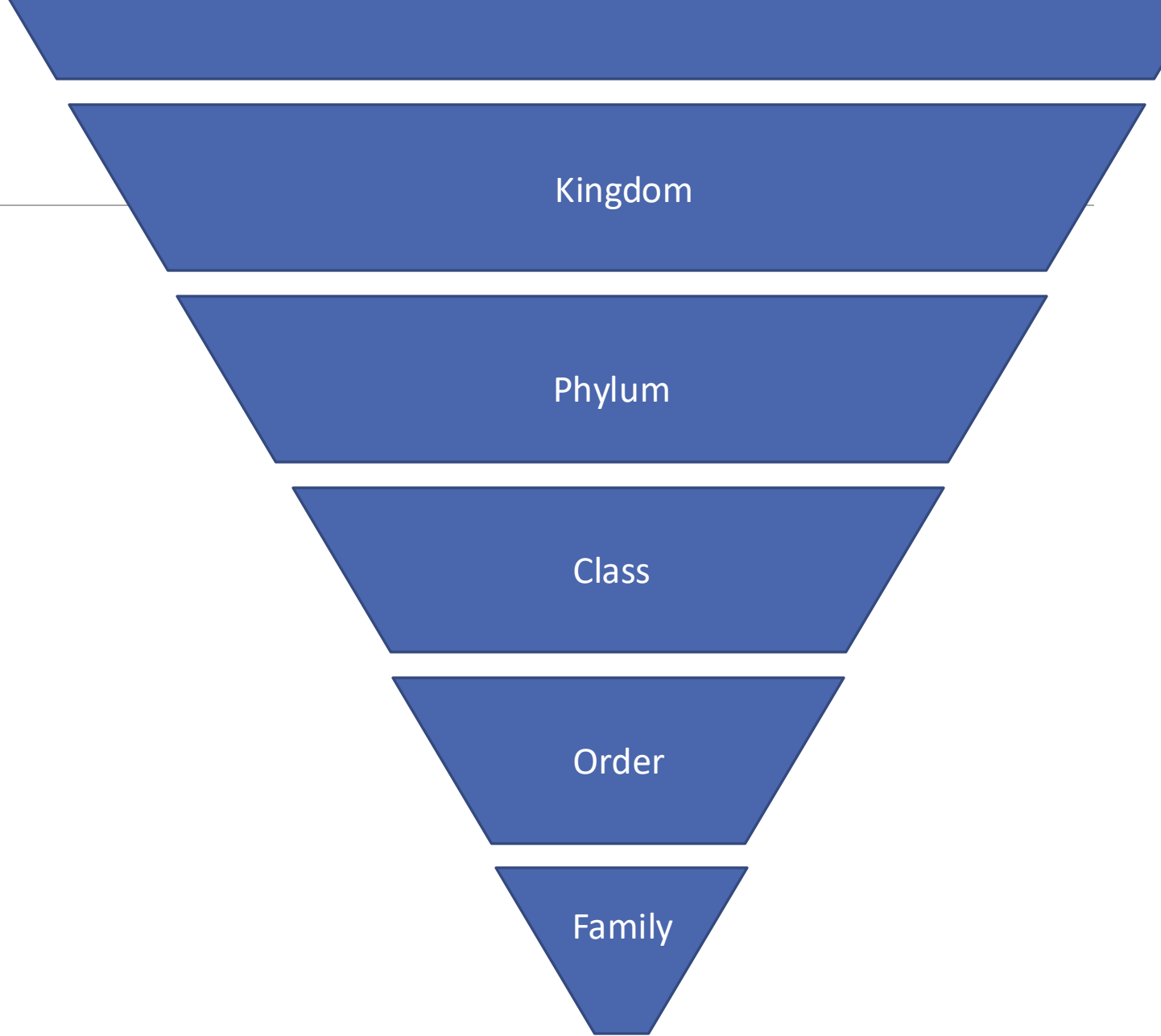
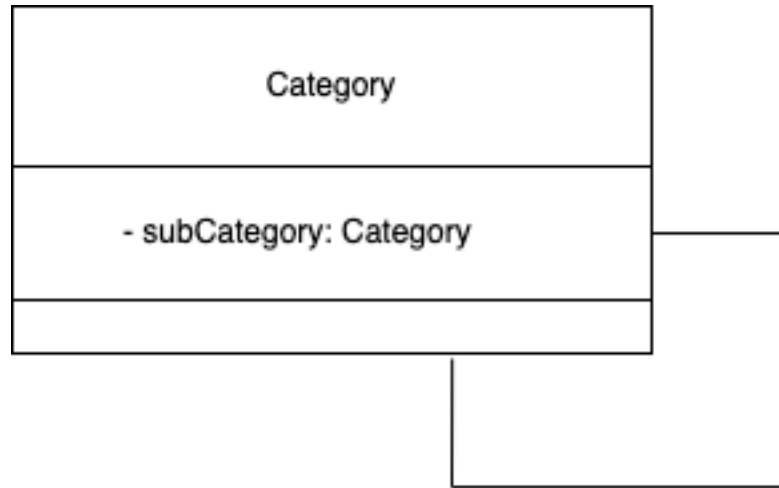
Cardinality

A Person can have between zero to three employers, inclusive

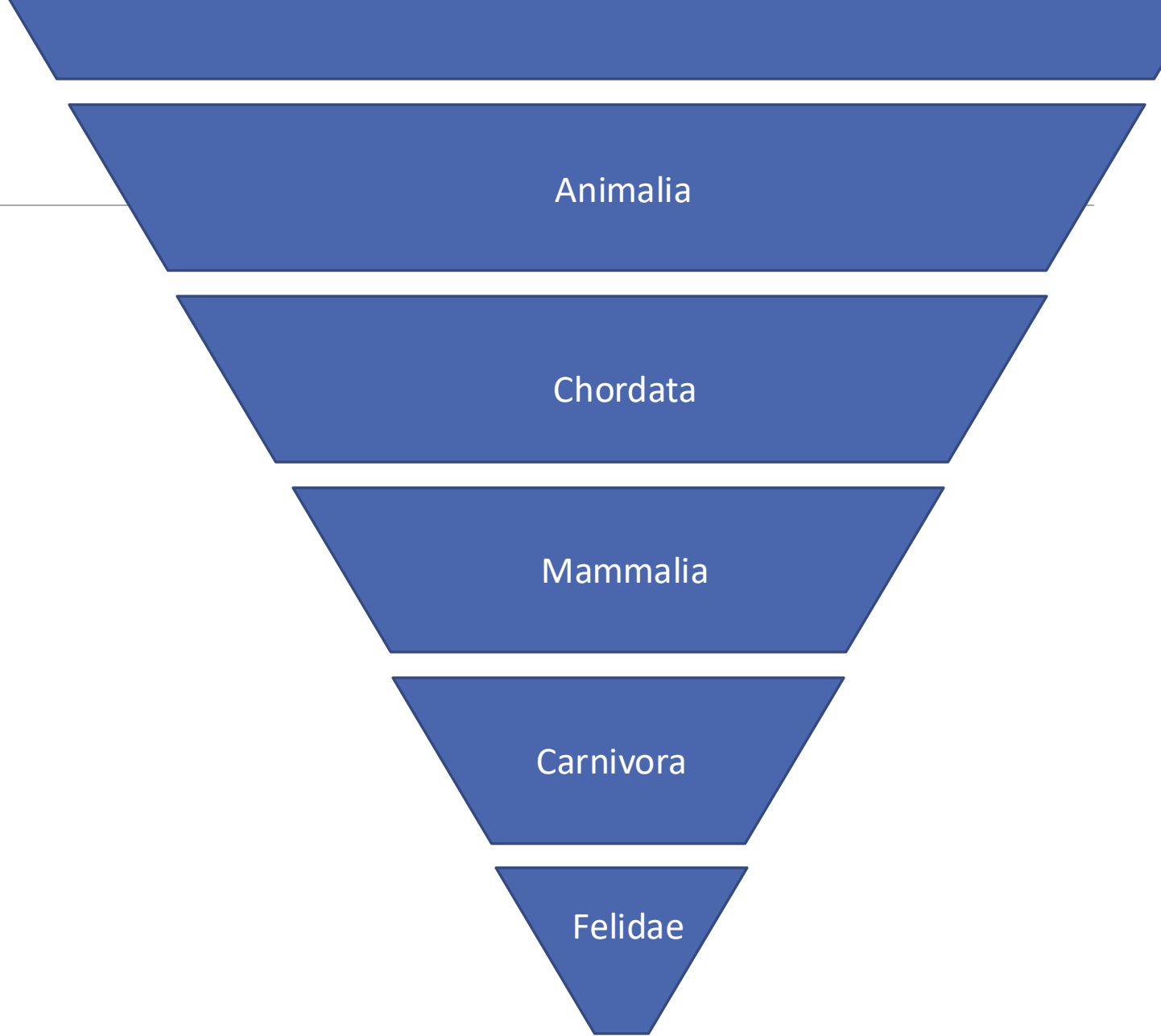
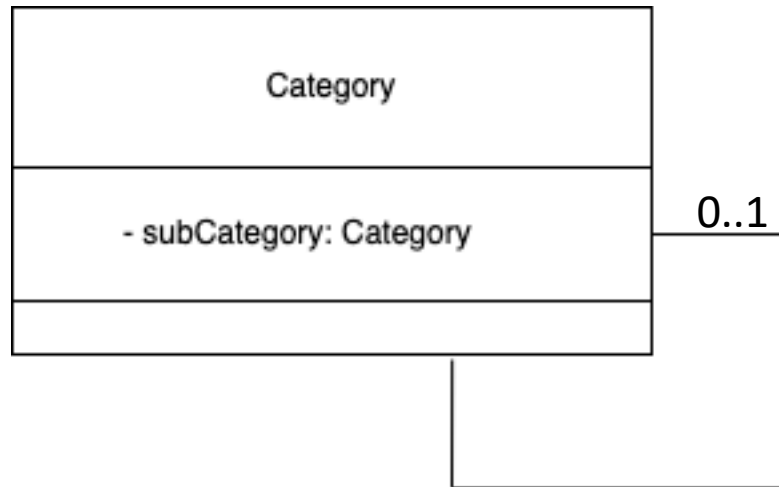
A Company must have at least two employees (but could have any number more)



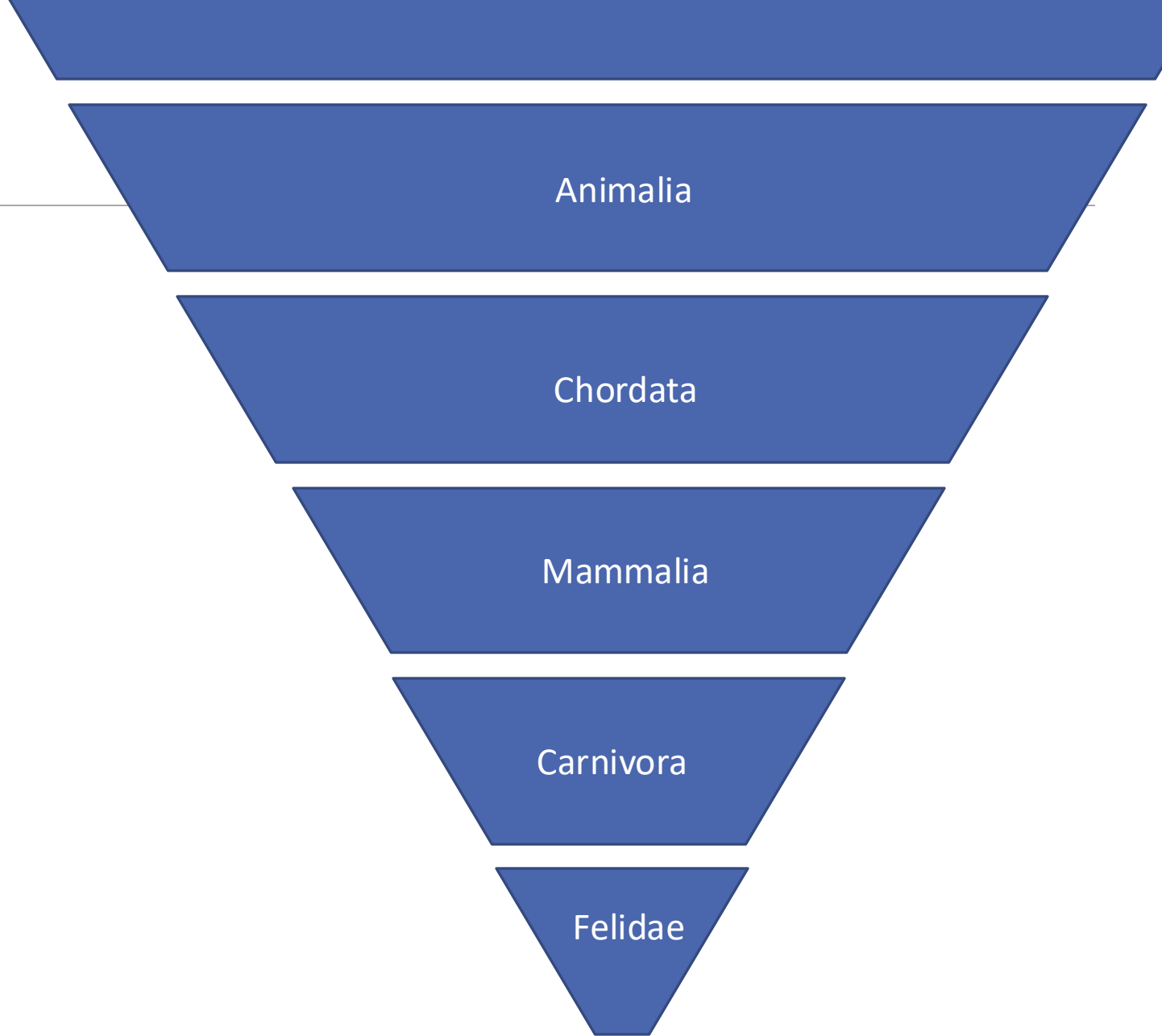
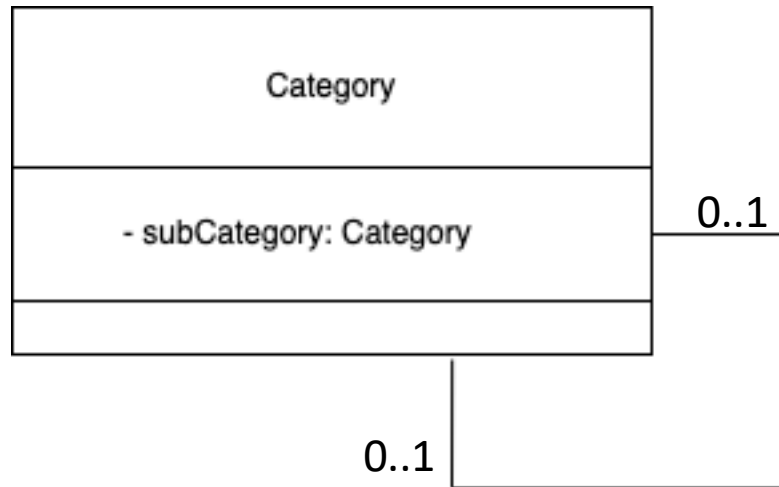
Reflexive Association



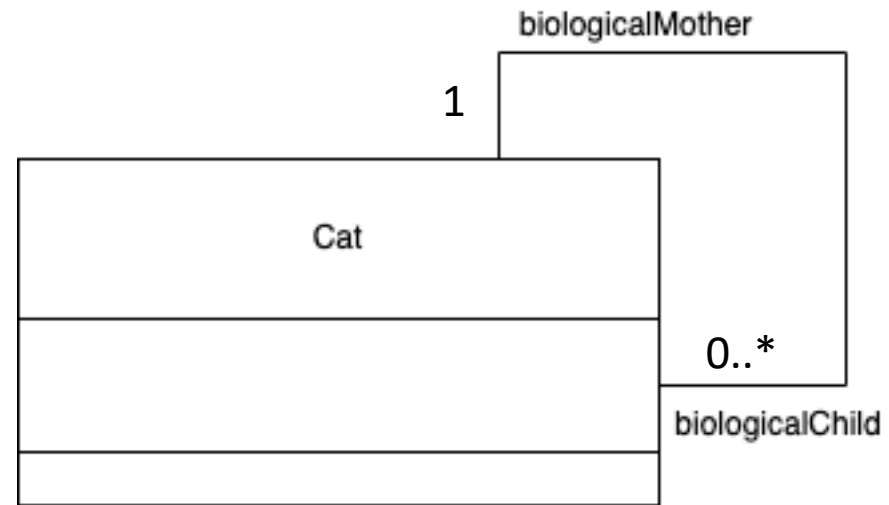
Reflexive Association



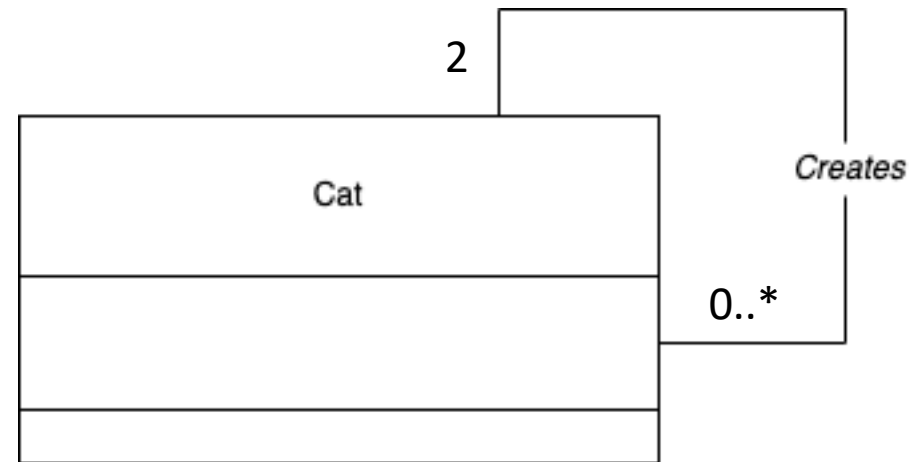
Reflexive Association



Reflexive Association



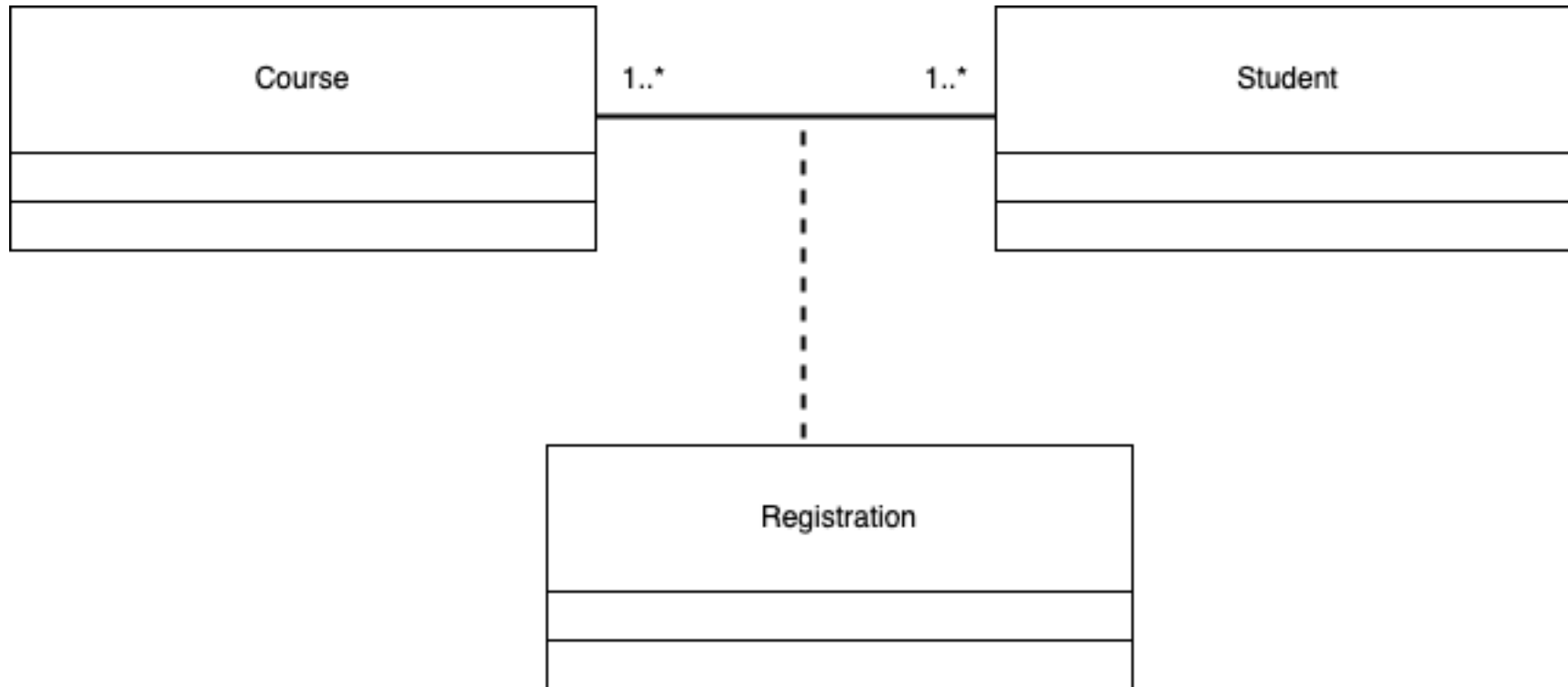
Reflexive Association



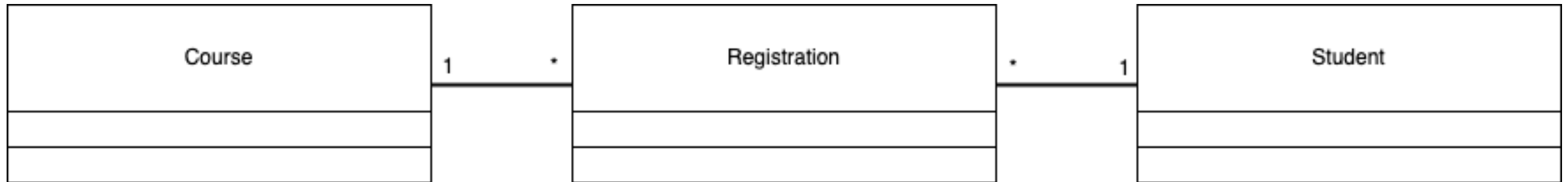
Exercise 16.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Association Class



Association Class



Aggregation

Expresses a connection between classes

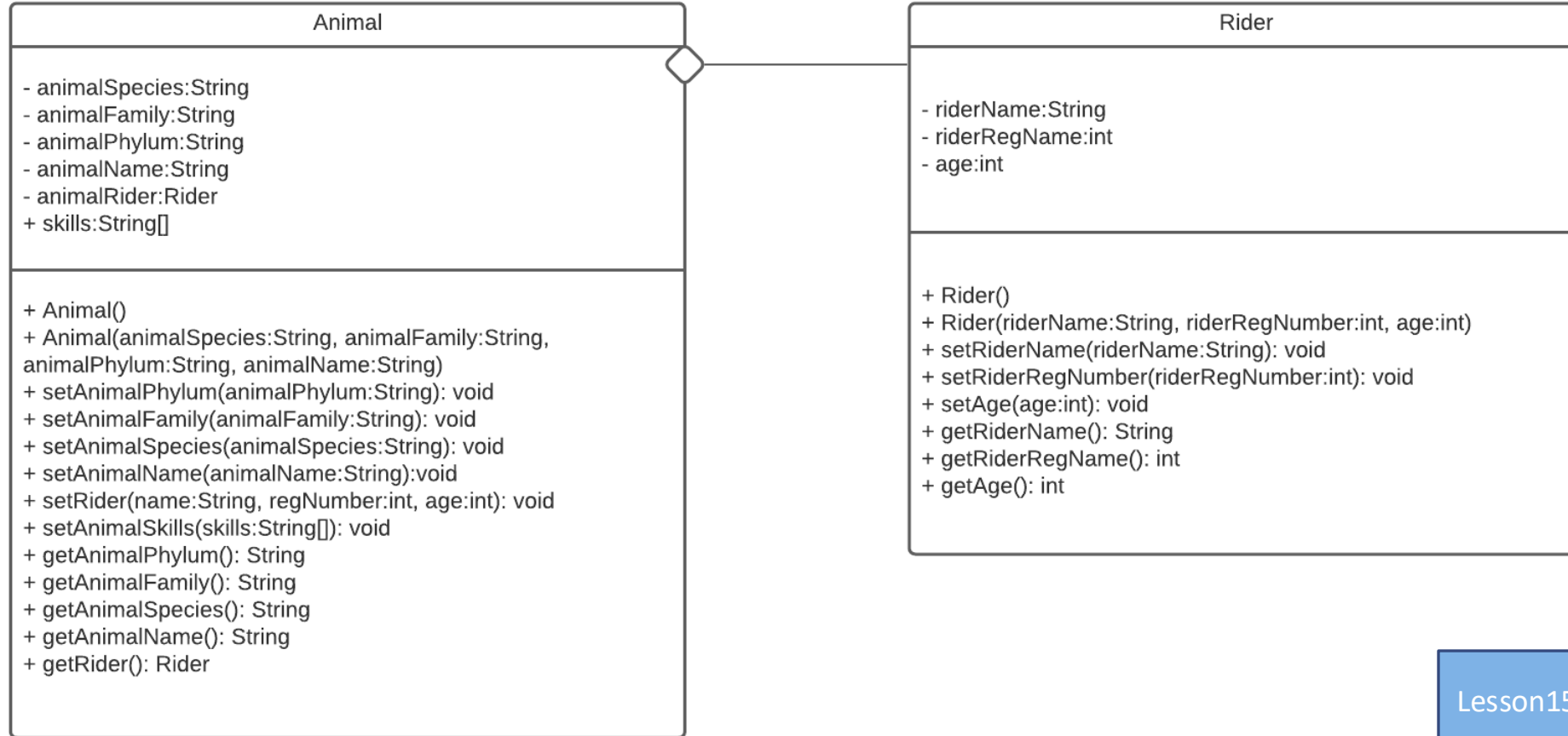
- Represents the "has a" relationship

The relationship between classes is asymmetric

- One class is a part of another

Aggregation

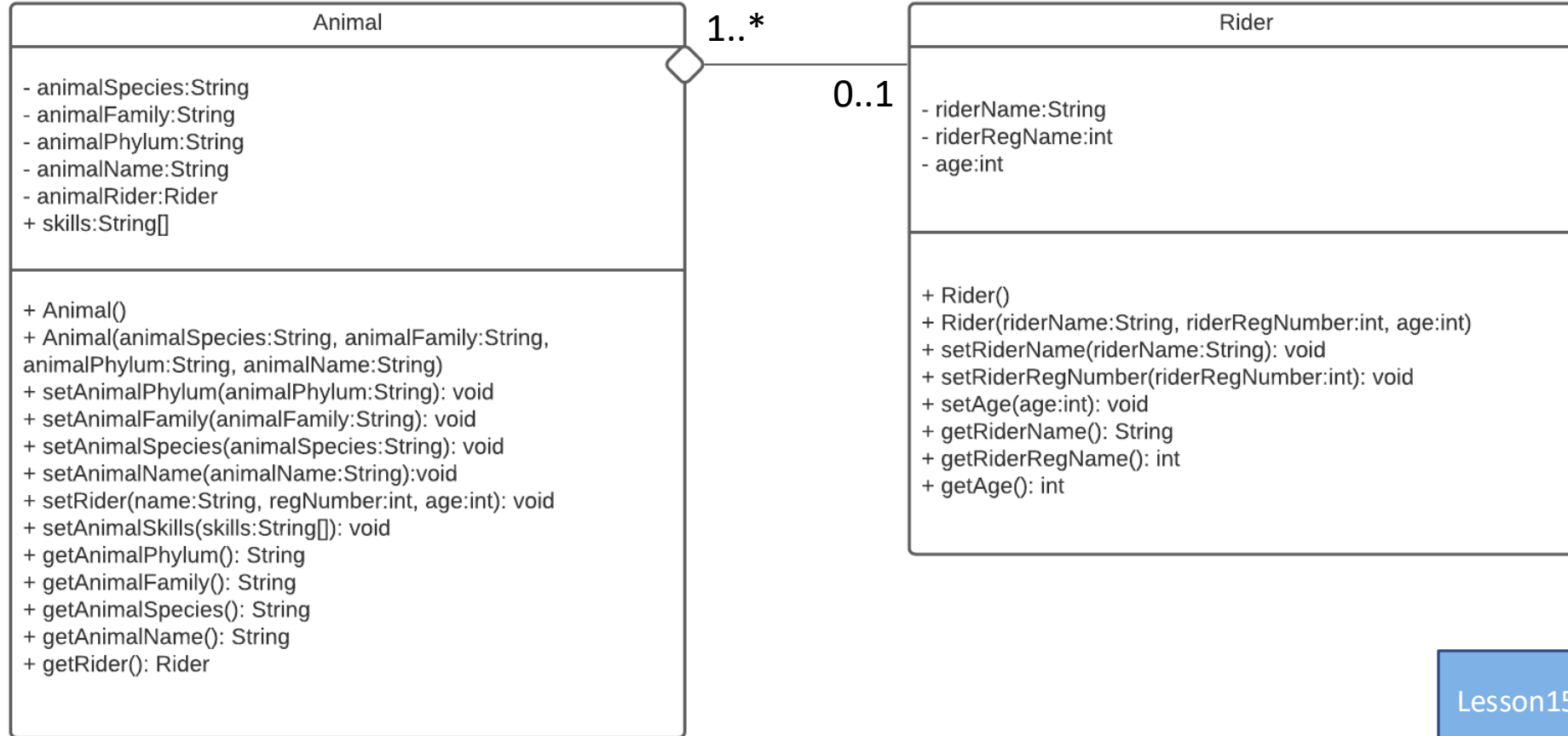
02_Aggregation



Lesson15/03_DeepCopy

Aggregation

02_Aggregation



Lesson15/03_DeepCopy

Aggregation

```
Animal horse1 = new Animal("Horse","Equidae","Chordata","Blaze");  
horse1.setRider("Alex", 4321, 30);
```

```
private Rider animalRider = new Rider();
```

```
public void setRider(String name, int regNumber, int age)  
{  
    this.animalRider.setRiderName(name);  
    this.animalRider.setRiderRegNumber(regNumber);  
    this.animalRider.setAge(age);  
}
```

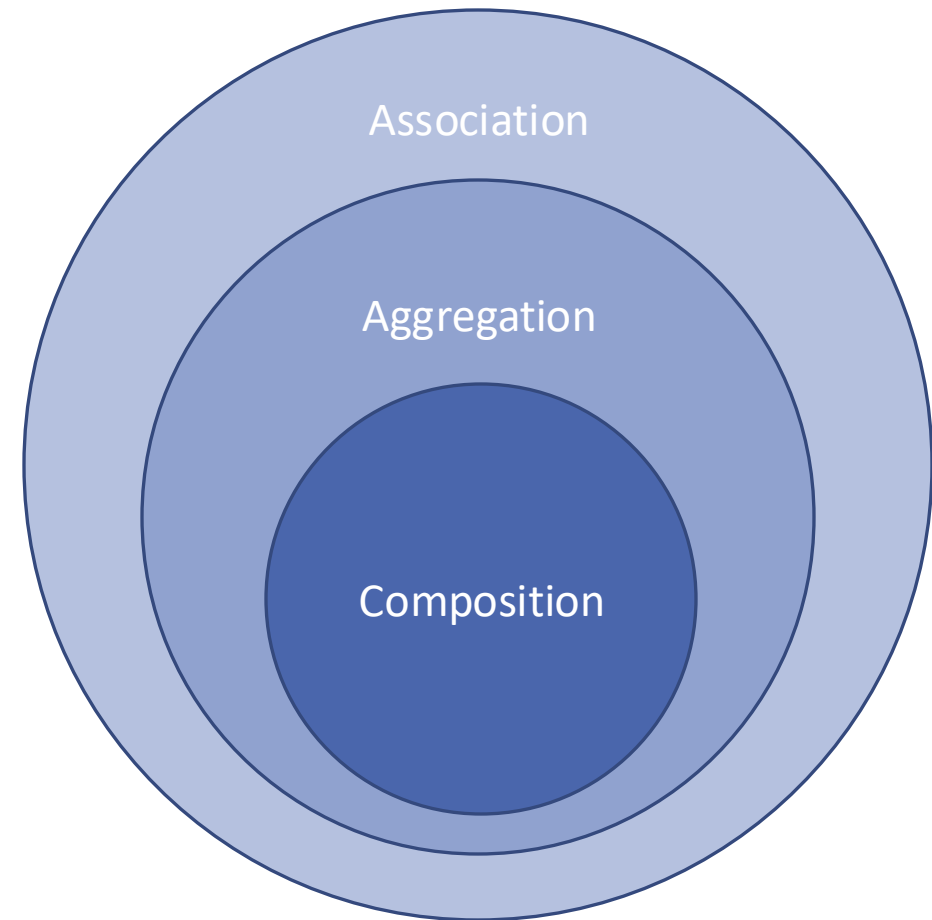

Composition

Expresses a connection between classes

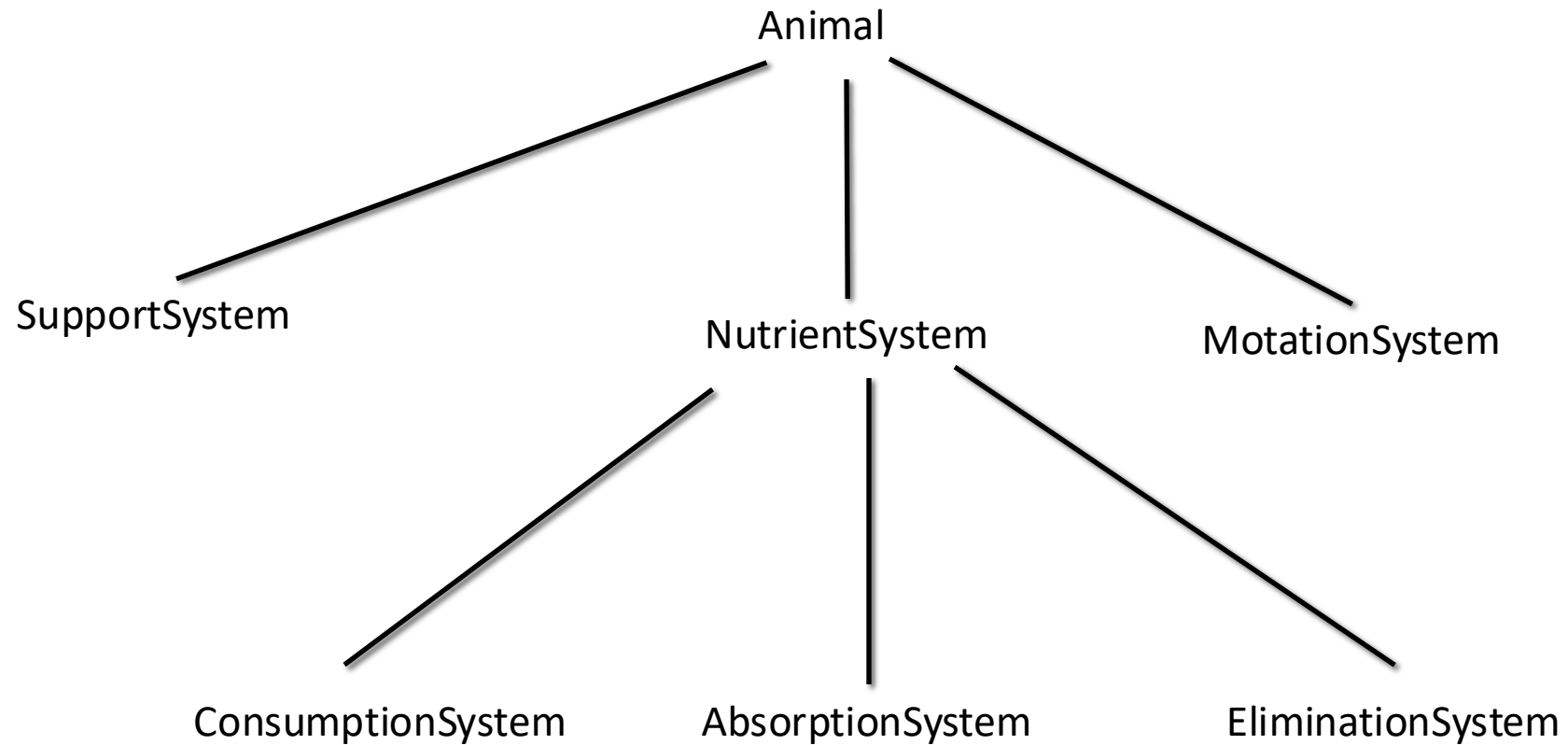
- Represents the "has a" relationship

The relationship between classes is asymmetric

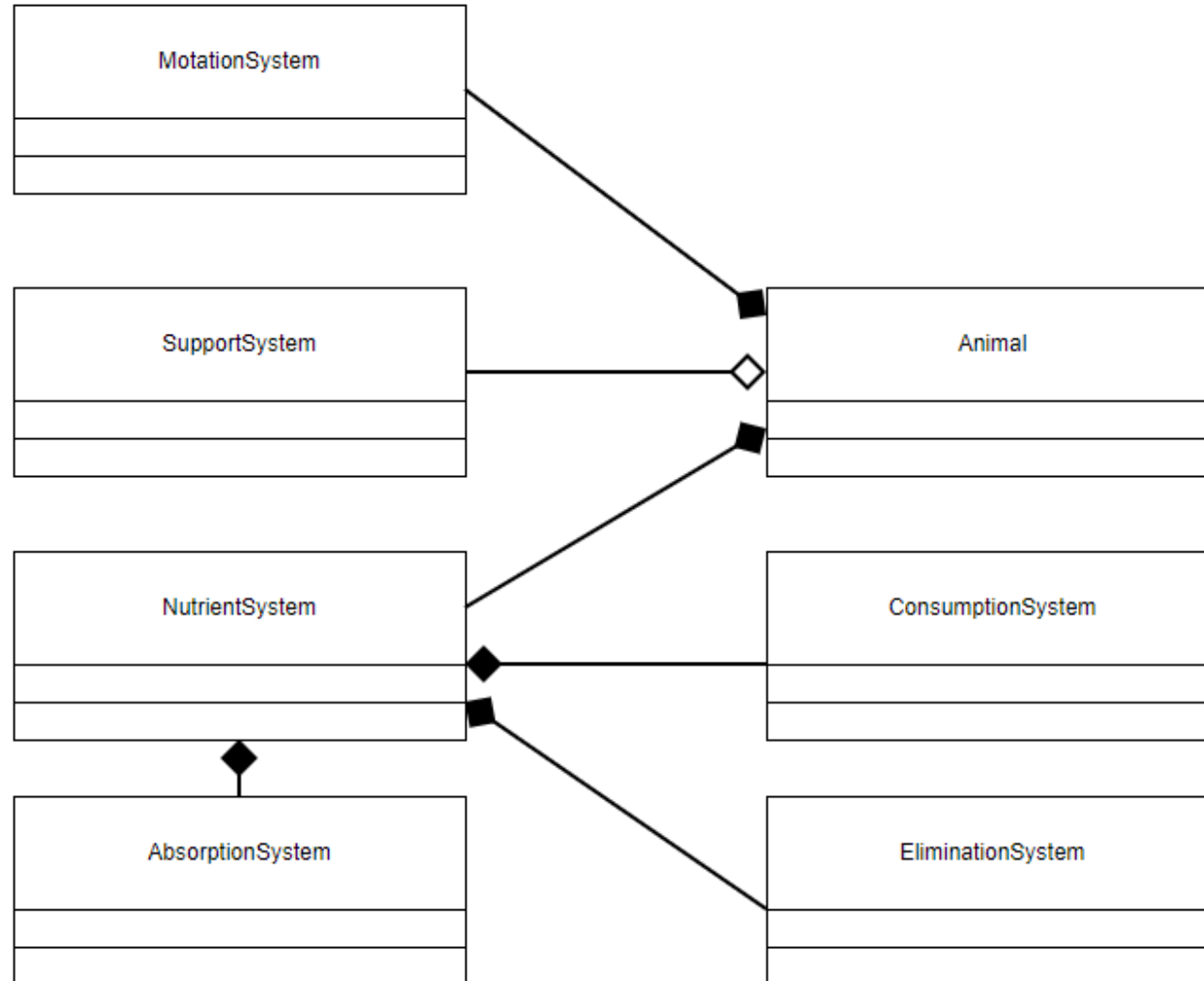
- One class is a part of another
- **Deletion of the whole destroys the part**



Composition



Composition



Exercise 16.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Association vs Aggregation vs Composition

Association

- “uses” relationship
- No implied ownership
- Represented with a line (bidirectional) or arrow (unidirectional)
- Example: a person has a partner

Aggregation

- “has a” relationship
- Destruction of the “whole” does not destroy the part, a weak form of ownership
- Represented with an open diamond arrow
- Example: a car has an engine

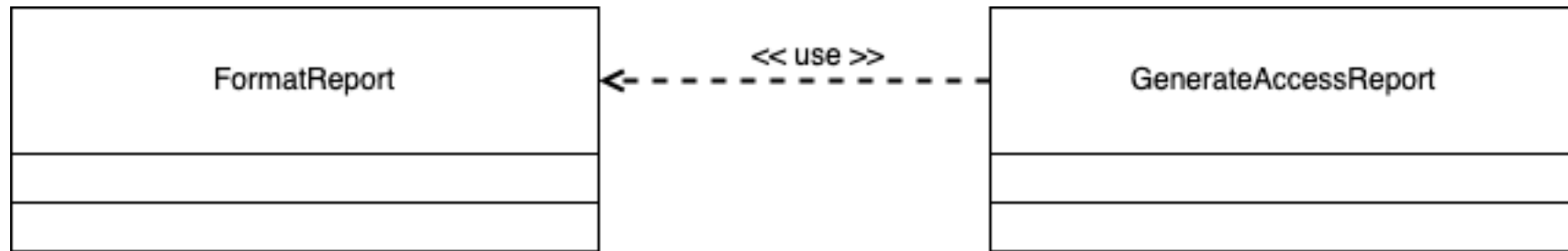
Composition

- “has a” relationship
- Destruction of the “whole” destroys the “part”, a strong form of ownership
- Represented with a solid diamond arrow
- Example: a house has rooms

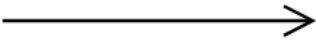
Dependency

A temporary reliance or dependency

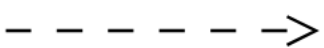
Between two classes, one relationship is a "client" and "service" relationship, or “references”



Association vs Dependency

Association 

- “uses” relationship
- Represented with a line (bidirectional) or arrow (unidirectional)
- There is a conceptual link between the two objects
- One (unidirectional) or both (bidirectional) objects store the other object as an attribute
- Example: a person has a partner

Dependency 

- “references” relationship – class A “depends upon” class B to function
- Represented by a dashed line with an arrow (usually with the dependency named)
- There is no conceptual link between the two objects
- Use of the object is temporary, the object is not stored – for example one object accepts another as a parameter to a method, or instantiates an object when needed
- Example: a payment gateway depends on a user’s credit card

Exercise 16.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Association

- Directionality (uni- and bi-)
- Reflexive
 - Asymmetric
 - Symmetric
 - Roles
- Name of association

Association Class

Aggregation

Composition

Dependency

Cardinality (multiplicity)

Summary: Skills

Depict the following in a UML class diagram

- Association, including reflexive association
- Association class
- Aggregation
- Composition
- Use dependency
- Cardinality

Additional Information

Related reading

- Volume 1, Chapter 4.1.4: Relationships between Classes
- Volume 1, Chapter 4.1.3: Identifying Classes